

Tema 2

Sincronización en memoria compartida

SCD para GIIM-ADE

Asignatura *Sistemas Concurrentes y Distribuidos*

Fecha 1 de octubre 2025



Monitores como
mecanismo de alto
nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Departamento de Lenguajes y Sistemas Informáticos
Universidad de Granada

Limitaciones de los semáforos

Hay algunos inconvenientes respecto de usar mecanismos como los semáforos para desarrollar aplicaciones concurrentes:

- ✗ Basados en variables globales
- ✗ El uso y función de las (variables *protegidas*) de los semáforos) rompe con los principios de desarrollo de código estructurado en los programas
- ✗ Operaciones se encuentran dispersas y no protegidas en el código

Por tanto, es necesario:

- ✓ un nuevo *mecanismo* de programación que permita la encapsulación de la información y de la sincronización entre procesos,
- ✓ programar las operaciones de sincronización (`wait`, `signal post...`) dentro de bloques o procedimientos que se ejecuten como instrucciones *atómicas*



Monitores como mecanismo de alto nivel

- Definición de monitor
- Funcionamiento de los monitores
- Sincronización en monitores
- Verificación de monitores
- Patrones de solución con monitores
- Colas de prioridad
- Semántica de las señales de los monitores
- Implementación de los monitores

Concepto

C.A.R. Hoare, en 1974, idea el concepto de Monitor, que es un mecanismo de programación de alto nivel que permite definir objetos abstractos compartidos entre los procesos concurrentes



Monitores como
mecanismo de alto
nivel

Idea fundamental

En los monitores de Hoare, las variables internas y los procedimientos están siempre protegidos por exclusión mutua. Esto significa que solo un proceso puede usarlos a la vez, evitando interferencias. Además, las operaciones de sincronización no se hacen de manera externa ni dispersa, sino que quedan encerradas dentro del propio monitor. De esta forma, el programador no tiene que preocuparse de gestionar la sincronización cada vez: basta con llamar a los procedimientos del monitor, y este se encarga automáticamente de aplicar las reglas de exclusión mutua y de coordinación necesarias.

Definición de monitor

- Funcionamiento de los monitores
- Sincronización en monitores
- Verificación de monitores
- Patrones de solución con monitores
- Colas de prioridad
- Semántica de las señales de los monitores
- Implementación de los monitores

Monitor

“Módulo” de las aplicaciones concurrentes

- ✓ **Modularidad** en el desarrollo de programas y aplicaciones
- ✓ **Programa**= {*Monitores, Procesos concurrentes*}
- ✓ **Estructuración** en el acceso a tipos de datos, variables compartidas, etc.
- ✓ **Capacidad de modelado** de interacciones cooperativas y competitivas entre procesos concurrentes, lo más general posible
- ✓ **Ocultación** a los procesos de las operaciones de sincronización sobre datos compartidos
- ✓ **Reusabilidad** basada en parametrización de los módulos monitor
- ✓ **Verificación formal con el sistema de reglas** mediante reglas más simples que las de los semáforos



Monitores como
mecanismo de alto
nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Semántica de los componentes de un monitor



Monitores como
mecanismo de alto
nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

- ✓ **Exclusión mutua** en el acceso a los procedimientos/métodos del módulo monitor
- ✓ **Variables permanentes:** son el estado interno del monitor
 - Sólo pueden ser accedidas dentro del monitor (en el cuerpo de los procedimientos y código de inicialización)
 - Permanecen sin modificaciones entre dos llamadas consecutivas a procedimientos del monitor
- ✓ **Procedimientos:** modifican el estado interno (garantizando la exclusión mutua durante dicho cambio)
 - Pueden tener variables y parámetros locales, que toman un nuevo valor en cada activación del procedimiento
 - Constituyen la interfaz externa del monitor y podrán ser llamados por los procesos que comparten el recurso
- ✓ **Código de inicialización:** fija el estado interno inicial
 - Este bloque es opcional
 - Se ejecuta una única vez, antes de cualquier llamada a procedimientos del monitor

Diagrama de los componentes del monitor



Monitores como mecanismo de alto nivel

Definición de monitor

- Funcionamiento de los monitores
- Sincronización en monitores
- Verificación de monitores
- Patrones de solución con monitores
- Colas de prioridad
- Semántica de las señales de los monitores
- Implementación de los monitores

Monitor

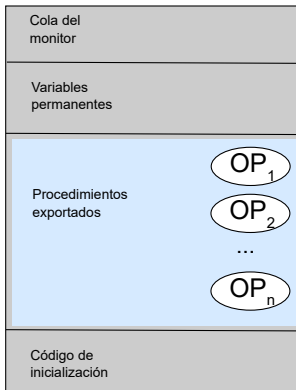


Figura: Un monitor se puede visualizar como aparece en el diagrama

Concepto de *monitor* III



Monitores como
mecanismo de alto
nivel

Definición de monitor

- Funcionamiento de los monitores
- Sincronización en monitores
- Verificación de monitores
- Patrones de solución con monitores
- Colas de prioridad
- Semántica de las señales de los monitores
- Implementación de los monitores

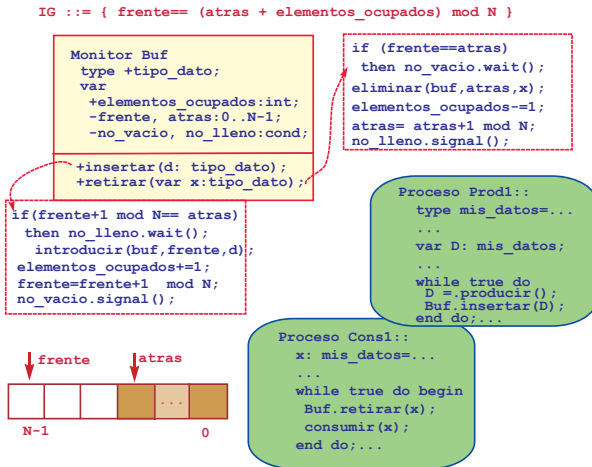


Figura: Representación de gráfica del monitor Búfer

Características de la programación con *monitores*

- Centralización de recursos críticos
- Sólo 1 procedimiento **ejecutado por un solo proceso**
- Los procedimientos pueden **interrumpirse**
- Posibilidad de **ejecución concurrente de monitores no-relacionados**

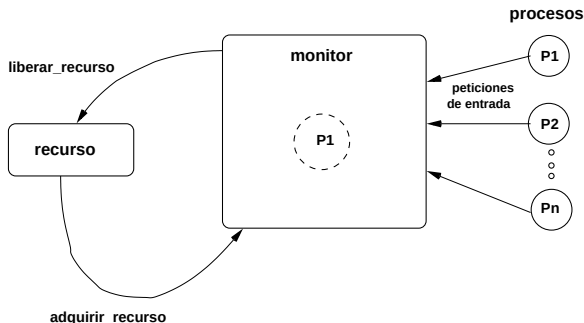


Figura: Representación gráfica de alto nivel de un módulo monitor



Monitores como
mecanismo de alto
nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Instanciación de los monitores

¿Qué ocurre tras crear un monitor?

- Un monitor es un **objeto pasivo**: su código solo se ejecuta cuando un proceso invoca alguno de sus procedimientos



Monitores como
mecanismo de alto
nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

¿Por qué instanciar monitores?

- Para obtener **copias independientes** del recurso gestionado por el monitor.
- Así, varios procesos pueden trabajar de forma concurrente sin interferirse

Analogía práctica

- Igual que en un *sistema de ficheros distribuido (DCE-DFS)*, donde diferentes usuarios acceden a recursos compartidos como si fueran locales.
- La instanciación de monitores permite un **uso transparente y concurrente** de los recursos



Monitores como
mecanismo de alto
nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

- Cada instancia tiene sus variables permanentes propias
- La exclusión mutua (EM) en el acceso a los recursos protegidos por el monitor ocurre en cada instancia por separado

Condición para que un lenguaje de programación pueda compilar monitores instanciables:

El código de los procedimientos de los monitores ha de ser *reentrante*, es decir:

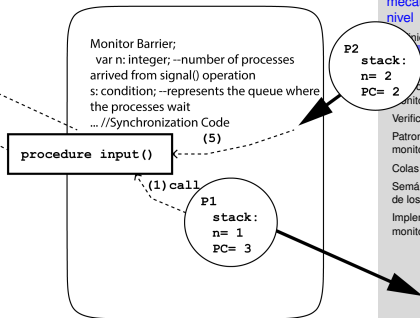
- No dependerá de datos estáticos globales
- No puede modificar su propio código
- No puede llamar a funciones *no-reentrantes*

Ejemplo de procedimiento reentrante llamado por procesos concurrentes



The reentrancy assures that each process: P1, P2, sees their own copies of variable n and PC

```
procedure input(); (2) (6) ←  
(1) begin  
(2) n:= n + 1; ←one more process (7) ↓  
arrived  
(3) if n < 2 then --it is the  
first process (3) ↓  
(4) s.wait(); --wait for the  
second process  
(3') else begin --is the second  
process (8) ↓  
(4') n:= 0; --initiates the count (9) ↓  
(5') s.signal() --awakes the first  
process  
end  
end;
```



Monitores como mecanismo de alto nivel

- Definición de monitor
- Manejamiento de los procesos
- Sincronización en monitores
- Verificación de monitores
- Patrones de solución con monitores
- Colas de prioridad
- Semántica de las señales de los monitores
- Implementación de los monitores

Instanciación de los monitores-III

```
class monitor VariableProtegida(entr, salid : integer);  
var x, inc : integer;  
// incremento(), valor(); son los procedimientos llamables  
procedure incremento( );  
begin  
    x := x+inc ;  
end;  
procedure valor(var v : integer);  
begin  
    v := x ;  
end;  
begin  
x:= entr ; inc := salid ;  
end
```

mv1 y mv2 pueden ser compartidas por varios procesos concurrentes sin que se produzca *interferencia* (*condiciones de carrera* al acceder al recurso del monitor).

```
var mv1 : VariableProtegida(0,1);  
mv2 : VariableProtegida(10,4); i1,i2 : integer ;  
begin  
mv1.incremento( ) ;  
mv1.valor(i1) ; { i1==1 } //permanentes (x,inc) distintas  
mv2.incremento( ) ; //para cada instancia del monitor  
mv2.valor(i2) ; { i2==14 }  
end
```



Monitores como
mecanismo de alto
nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Idea clave

El monitor controla la **exclusión mutua** gracias a una **cola de entrada** donde esperan los procesos que quieren acceder.

- Cuando un proceso está **ejecutando un procedimiento** dentro del monitor, los demás que intentan entrar se bloquean y se colocan en la cola.
- Al terminar, el proceso que estaba dentro **libera el monitor**.
- Entonces, el **primer proceso en la cola** (FIFO) es desbloqueado y entra al monitor.
- Si la cola está vacía, el monitor queda libre esperando nuevas llamadas.

Importante

La política FIFO garantiza la **vivacidad**: todos los procesos en espera acabarán entrando en el monitor.



Monitores como
mecanismo de alto
nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Diagrama de estados de un proceso



Monitores como mecanismo de alto nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

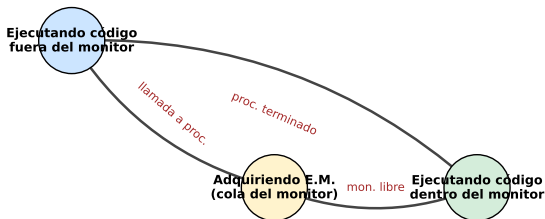


Figura: Posibles estados de los procesos y las transiciones entre dichos estados

Estado del monitor



Monitores como
mecanismo de alto
nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

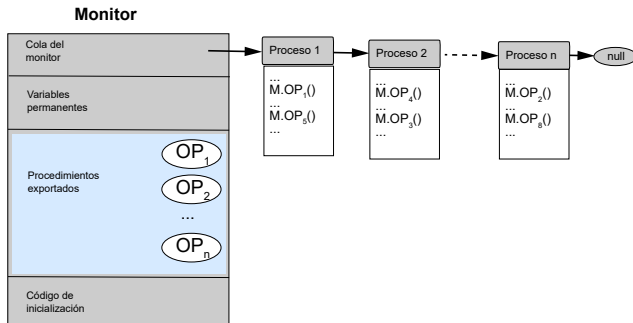
Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores



El estado del monitor incluye la cola de procesos esperando a comenzar a ejecutar el código del mismo

Operaciones de sincronización en monitores

Para implementar la sincronización, se requiere de una facilidad para que los procesos hagan esperas bloqueadas, hasta que sea cierta determinada condición

En semáforos existe:

- La posibilidad de bloqueo (`sem_wait()`) y activación (`sem_post()`)
- Un valor entero (el valor de la variable protegida “s” del semáforo), que indica si la condición se cumple (variable: > 0) o no ($= 0$).

En monitores, sin embargo, solo se dispone de:

- ✓ Sentencias de bloqueo y activación de la ejecución de una hebra de código
- ✓ No existe ninguna variable, ni valor protegido, asociado a 1 variable condición

Los valores de las variables permanentes del monitor determinan si se cumple o no se cumple la *condición de sincronización* de los procesos



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los monitores

Sincronización en monitores

Verificación de monitores
Patrones de solución con monitores
Colas de prioridad
Semántica de las señales de los monitores
Implementación de los monitores

Operaciones de sincronización en monitores-II

Encapsulación de la sincronización

- Explícitamente programada dentro de los procedimientos
- TDA `cond`
- Variables condición (sólo declararlas, no inicializarlas)
- Se define 1 variable condición, por cada una de las condiciones para esperar, en el monitor

<code>c.wait()</code> : bloquea siempre. El desbloqueo de los procesos se produce en orden FIFO	<code>c.signal()</code> : si la cola de <code>c</code> no está vacía, desbloquea al primer proceso de la cola
---	---

- Las variables condición pertenecen a un tipo abstracto y opaco de datos La representación interna de las variables condición no es accesible al programador de monitores
- La cola de procesos bloqueados asociada a 1 variable condición es FIFO

`c.queue()` :

función lógica que devuelve `true` si hay algún proceso esperando en la cola de `cond`, y `false` en caso contrario



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los monitores

Sincronización en monitores

Verificación de monitores
Patrones de solución con monitores
Colas de prioridad
Semántica de las señales de los monitores
Implementación de los monitores

El TDA *cond* de los monitores

- La EM del monitor se levanta como consecuencia de ejecutar `c.wait()` ¡cuidado!: deja “abierto” el monitor antes de bloquear al proceso
- Se cede el acceso del monitor al proceso señalado (`c.signal()` con semántica desplazante)
- La semántica desplazante de las señales evita un robo de señal por un tercer proceso que se adelante y entre en el monitor

Comportamiento de los procesos después de ejecutar las operaciones de sincronización

- Después de ejecutar `c.wait()`
- Después de ejecutar `c.signal()`

No pueden programarse operaciones `c.wait()` indebidas, ni tampoco omitirse las operaciones `c.signal` necesarias

- Operaciones `c.queue` y `c.signal_all`
- No es segura la simulación de `c.signal_all` utilizando `c.queue` y señales desplazantes



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los monitores

Sincronización en monitores

Verificación de monitores
Patrones de solución con monitores
Colas de prioridad
Semántica de las señales de los monitores
Implementación de los monitores

Estado de un monitor con varias colas



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los monitores

Sincronización en monitores

Verificación de monitores
Patrones de solución con monitores
Colas de prioridad
Semántica de las señales de los monitores
Implementación de los monitores

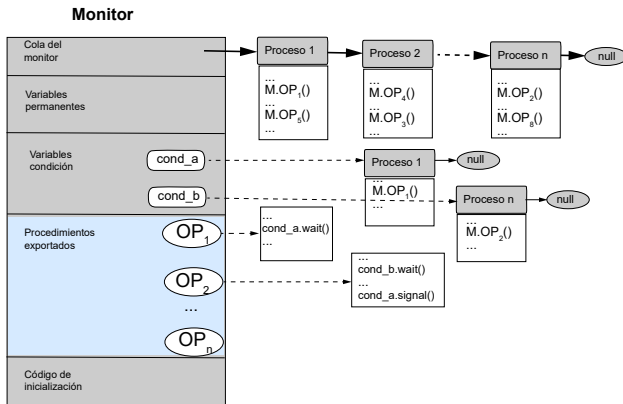


Figura: Los procesos 2 y n ejecutan las operaciones 1 y 2, que producen esperas de las condiciones *cond_a* y *cond_b*



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los
monitores
Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores
Colas de prioridad
Semántica de las señales
de los monitores
Implementación de los
monitores

El programador no puede conocer a priori la traza concreta de llamadas a los procedimientos del monitor por parte de los procesos del programa concurrente.

Monitor Buf;

Process P1 (consumidor) || Process P2 (consumidor) || Process P3(producer)

==>Trazas:

P3::Buf.insertar(x);P3::Buf.insertar(x');P2::Buf.retirar(x);P1::Buf.retirar(x'); ...

P3::Buf.insertar(x);P2::Buf.retirar(x);P1::Buf.retirar(x') [bloqueado]; P3::Buf.insertar(x');...

Invariante de un monitor (IM):

- Es una propiedad que el monitor cumple siempre, pero específico de cada monitor diseñado por un programador

Verificación de programas con monitores-II

Esquema general de demostración de programas con monitores:

- Probar la **corrección parcial de los procesos** secuenciales
- **Comprobar** que el **invariante** de cada uno de los monitores del programa se mantiene: inicialmente y antes/después de ejecutar cada procedimiento
- Aplicar la **regla de la concurrencia**



Características de un IM:

- ✓ Es una función lógica que se puede evaluar como *true* o *false* en cada estado del monitor a lo largo de la ejecución
- ✓ Su *valor de verdad* depende de los valores de las variables permanentes
- ✓ Debe ser cierto en cualquier estado del programa concurrente, excepto cuando un proceso está ejecutando código del monitor

Ejemplo de invariante del monitor Buf :

$IM:: frente = (atras + elementos_ocupados) \bmod N$

frente, *atras* y *elementos_ocupados* son las variables permanentes del monitor

Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los
monitores
Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores
Colas de prioridad
Semántica de las señales
de los monitores
Implementación de los
monitores

Verificación de los monitores

- Axiomas iniciales

La demostración de corrección se basa en:

Invariante del monitor(IM)

relación constante entre los valores permitidos de las variables permanentes del monitor

Ejemplo (del Monitor Buf) $\text{elementos_ocupados} \leq N$:

Axioma (inicialización variables)

$\{V\}$ inicialización variables permanentes $\{IM\}$

La inicialización se lleva a cabo dentro del **cuerpo** `begin ... end` del monitor

El invariante del monitor debe ser cierto en su estado inicial, justo después de la inicialización de las variables permanentes

Inicialmente: $\text{elementos_ocupados} = 0 \leq N$: dado que $N > 0$



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los monitores
Sincronización en monitores

Verificación de monitores

Patrones de solución con monitores
Colas de prioridad
Semántica de las señales de los monitores
Implementación de los monitores



El invariante del monitor debe ser cierto:

- ✓ Antes y después de cada llamada a un procedimiento del monitor

Axioma (procedimientos del monitor)

$$\{IN \wedge IM\} \text{procedimiento}_i \{OUT \wedge IM\}$$

- **IN** satisfecho por los parámetros *in*, *in/out* del procedimiento
- **OUT** satisfecho por los parámetros *out*, *in/out* del procedimiento

Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los
monitores
Sincronización en
monitores

Verificación de monitores

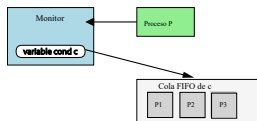
Patrones de solución con
monitores
Colas de prioridad
Semántica de las señales
de los monitores
Implementación de los
monitores

Axiomas de operaciones de sincronización con semántica desplazante

Axioma para la operación `c.wait()`

$$\{IM \wedge L\} \text{ c.wait() } \{C \wedge L\}$$

1. Cuando un proceso ejecuta `c.wait()`:
 - Se **bloquea** inmediatamente
 - **Libera** el monitor
2. El proceso entra en la **cola FIFO** asociada a `c`
3. En las demostraciones de corrección:
 - Se **ignora** el hecho de que el proceso debe reanudar tras `c.wait()`
 - Es decir, no debe quedar **indefinidamente suspendido**



El proceso que hace `c.wait()` entra en la cola FIFO de condición.



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los monitores
Sincronización en monitores

Verificación de monitores

Patrones de solución con monitores
Colas de prioridad
Semántica de las señales de los monitores
Implementación de los monitores

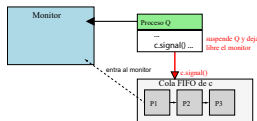
Axiomas de operaciones de sincronización con semántica desplazante - II



Axioma para la operación `c.signal()`

$$\{ \neg \text{vacio}(c) \wedge L \wedge \text{C} \} \text{ c.signal() } \{ \text{IM} \wedge L \}$$

1. Si la cola `c` está **vacía**: $\Rightarrow$ `c.signal()` **no tiene efecto**.
2. Con **semántica desplazante**: el monitor mantiene la condición **C** hasta que un proceso que estaba en `c.wait()` se reanude y continúe dentro del monitor.
3. Los axiomas **no especifican el orden** en que se desbloquean los procesos (si hay varios en la cola).



Un proceso de la cola de condición `c` se desbloquea cuando otro ejecuta `c.signal()`.

Monitores como mecanismo de alto nivel

- Definición de monitor
- Funcionamiento de los monitores
- Sincronización en monitores

Verificación de monitores

- Patrones de solución con monitores
- Colas de prioridad
- Semántica de las señales de los monitores
- Implementación de los monitores

Ejemplo: verificación con señales desplazantes

Regla verificación del IF: ambas ramas con misma postcondición $\{s > 0\}$

```
Monitor Semaforo;
```

```
var s: integer; {IM:  $s \geq 0$ }
```

```
condición de sincronización:  $\{s > 0\}$ 
```

```
c: cond;
```

```
procedure P;
```

```
begin
```

```
{IM}
```

```
if s=0 then
```

```
{ $s = 0 \wedge \text{IM}$ }
```

```
c.wait;
```

```
{ $s > 0$ }
```

```
else
```

```
{ $s > 0$ }
```

```
null;
```

```
{ $s > 0$ }
```

```
endif;
```

```
{ $s > 0$ }
```

```
s := s-1
```

```
{ $s \geq 0 \rightarrow \text{IM}$ }
```

```
end;
```

```
procedure V;
```

```
begin
```

```
{IM}
```

```
s := s+1;
```

```
{ $s > 0$ }
```

```
c.signal;
```

```
{ $s \geq 0 \rightarrow \text{IM}$ }
```

```
end;
```

```
begin
```

```
{TRUE}
```

```
s := 0;
```

```
{ $s \geq 0 \rightarrow \text{IM}$ }
```

```
end;
```



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Verificación de un monitor con señales desplazantes-II



Ejercicio: Semáforo de Habermann

$n_p \leq n_a$, ya que primero ha de incrementarse n_a
 $n_p \leq n_v$, ya que n_p no puede superar a n_v
 $n_p \geq \min(n_a, n_v)$, condición no necesaria, pero deseable para garantizar *vivacidad*

```
Monitor Semaforo;
```

```
  var na, np, nv:int  
      c: cond;
```

```
procedure P;  
begin
```

```
  na:= na+1;  
  if(na > nv)  
    then c.wait();  
  np:= np+1;
```

```
end;
```

```
procedure V;  
begin
```

```
  nv:= nv+1;  
  if(na > np)  
    then c.signal;  
end;
```

```
  na:=0;  
  nv:=0;  
  np:=0;
```

Monitores como
mecanismo de alto
nivel

Definición de monitor
Funcionamiento de los
monitores
Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores
Colas de prioridad
Semántica de las señales
de los monitores
Implementación de los
monitores

Verificación del Monitor “Semáforo de Habermann”

```
Monitor Semaforo;  
  var na, np, nv:int  
      c: cond;  
  procedure P();  
  begin{np==min(na,nv)}  
    na:= na+1;  
    {np==min(na-1,nv)}  
    if(na > nv)  
      then  
        {(na>nv) and np===min(na-1,nv)}=>  
          np== min(na,nv)  
          c.wait();  
          {na>np and np===nv-1}  
        else null  
          { np<nv and np===na-1}  
        endif  
        {np+1===min(na,nv)}  
        np:= np+1;  
        {np===min(na,nv)}  
      end;  
  {V}  na:=0;    nv:=0;    np:=0;    {np===min(na,nv)}
```

```
  procedure V();  
  begin {np==min(na,nv)}  
    nv:= nv+1;  
    {np==min(na,nv-1)}  
    if(na > np)  
      then  
        {(na>np) and np===min(na,nv-1)}=>  
          na>np and np== nv-1  
          c.signal;  
          {np==min(na,nv)}  
        end;  
  end;
```



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los
monitores
Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores
Colas de prioridad
Semántica de las señales
de los monitores
Implementación de los
monitores

Regla de la concurrencia para la verificación de programas con monitores

Idea general

Si cada proceso se puede verificar de forma aislada y no hay interferencia entre ellos, entonces el programa concurrente completo también es correcto.

$$\{P_i\} S_i \{Q_i\}, 1 \leq i \leq n$$

(1) Ninguna variable libre en P_i o Q_i es modificada por S_j , $i \neq j$

(2) Todas las variables en IM_k son locales al monitor m_k

$$\begin{array}{c} \{IM_1 \wedge \dots \wedge IM_m \wedge P_1 \wedge \dots \wedge P_n\} \\ \text{cobegin } S_1 \parallel S_2 \parallel \dots \parallel S_n \text{ coend} \\ \{IM_1 \wedge \dots \wedge IM_m \wedge Q_1 \wedge \dots \wedge Q_n\} \end{array}$$

- La concurrencia con monitores **elimina la interferencia**: cada proceso se verifica como si fuera secuencial.
- Si los triples de Hoare individuales $\{P_i\} S_i \{Q_i\}$ no son críticos (no se interfieren), entonces sus pruebas **cooperan automáticamente**, sin necesidad de demostrar **teoremas de no-interferencia**.



Se estudian a continuación los patrones de solución para tres problemas sencillos, típicos de la Programación Concurrente

- Espera única (EU): un proceso, antes de ejecutar una sentencia, debe esperar a que otro proceso complete otra sentencia (ocurre típicamente cuando un proceso debe leer una variable escrita por otro proceso, el primero se suele denominar *consumidor* y el segundo *productor*)
- Exclusión mutua(EM): acceso en exclusión mutua a una sección crítica por parte de un número arbitrario de procesos
- Problema del productor/consumidor(PC): similar a la espera única, pero de forma repetida en un bucle (un proceso *productor* escribe sucesivos valores en una variable, y cada uno de ellos debe ser leído una única vez por otro proceso *consumidor*)



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Uso del monitor de EU

```
//variables compartidas
var x: integer; //contiene cada valor producido
process Productor; //escribe x
  var a: integer;
  begin
    a:=ProducirValor();
    x:=a; //sentencia E
    EU.notificar(); //sentencia N
  end
process Consumidor //lee x
  var b: integer;
  begin
    EU.esperar(); //sentencia W
    b:=x; //sentencia L
    EU.termina();
    UsarValor(b);
  end
end
```

- El proceso *consumidor* espera, antes de leer, a que el Productor termine la sentencia de escritura
- De esta forma nos aseguramos que es posible un entrelazamiento **E, L**, pero no puede ocurrir el entrelazamiento **L, E**



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los monitores

Sincronización en monitores

Verificación de monitores

Patrones de solución con monitores

Colas de prioridad

Semántica de las señales de los monitores

Implementación de los monitores

Monitor para Espera Única (EU)

```
monitor EU;  
  var terminado:boolean;  
  //terminado==true si fin de calculo, si no:terminado==false  
  leer_autorizado: boolean //variable auxiliar  
  cola:condition;  
  //cola consumidor esperando a que terminado==true  
export esperar, notificar;  
  //Invariante: terminado= false => lee_autorizado= false  
procedure esperar(); //para llamar antes de sentencia L  
begin  
  if (not terminado) then //si no se ha terminado W  
    cola.wait(); //esperar hasta que termine  
  //Condicion de sincronizacion terminado= true  
  leer_autorizado:= true; //funciona solo suponiendo  
    semantica desplazante de la senial!!!  
end;  
...
```

Un proceso (consumidor) no puede ejecutar una sentencia L (típicamente una lectura) hasta que otro proceso (productor) no acabe otra sentencia E (típicamente una escritura)



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los monitores

Sincronización en monitores

Verificación de monitores

Patrones de solución con monitores

Colas de prioridad

Semántica de las señales de los monitores

Implementación de los monitores

Monitor para Espera Única (EU)-II



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

```
...  
procedure termina();  
begin  
    terminado:= false;  
    leer_autorizado:= false;  
end;  
procedure notificar();//para llamar despues de E  
begin  
    terminado:=true;//Condicion de sincronizacion  
    cola.signal();//reactivar al otro, si esta esperando  
end  
begin { inicializacion: }  
    terminado := false;//al inicio no ha terminado E  
    leer_autorizado:= false;  
end;
```



Monitores como mecanismo de alto nivel

- Definición de monitor
- Funcionamiento de los monitores
- Sincronización en monitores
- Verificación de monitores
- Patrones de solución con monitores
- Colas de prioridad
- Semántica de las señales de los monitores
- Implementación de los monitores

Invariante : $terminado = false \Rightarrow leer_autorizado = false$

```
procedure esperar();
{Invariante, terminado= false, leer_autorizado= false}
if (not terminado) then
  begin
    {terminado= false, leer_autorizado= false, Invariante}
    cola.wait();
  endif;
  {Condicion de sincronizacion: terminado= true}
  leer_autorizado:= true
  {Invariante, se cumple porque se hace falso el
   antecendente}
end;
```

```
procedure notificar();
begin
  {terminado= false, leer_autorizado=false, Invariante}
  terminado:= true;
  {Condicion de sincronizacion: terminado= true}
  cola.signal();
  {Invariante, cuando vuelve el control aqui, se cumple}
end;
```

El monitor puede ser utilizado por n procesos concurrentes:

```
process Usuario[ i : 0..n ]
begin
  while true do begin
    EM.entrar(); //esperar SC libre, registrar SC ocupada
    ..... //seccion critica (SC)
    EM.salir(); //registrar SC libre, senialar
    ..... //otras actividades (RS)
  end
end
```

- #E es el número de llamadas a entrar completadas
- #S es el número de llamadas a salir completadas
- El número de procesos en SC es $\#E - \#S$
- El único entrelazamiento correcto es **E, S, E, S,...**
- Se debe cumplir $0 \leq \#E - \#S \leq 1$



Monitores como
mecanismo de alto
nivel

Definición de monitor
Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Patrón de Exclusión Mutua (EM)

```
monitor EM ;
  var ocupada: boolean;
    //ocupada==true hay un proceso en SC,
    //si no: ocupada==false
  cola: condition;
    //cola de procesos esperando a que ocupada==false
  export entrar, salir;
    //nombra procedimientos publicos
procedure entrar(); //protocolo de entrada (sentencia E)
begin
  if ocupada then //si hay un proceso en la SC
    cola.wait(); //esperar hasta que termine
    ocupada:=true; //indicar que la SC est ocupada
  end
procedure salir(); //protocolo de salida (sentencia S)
begin
  ocupada := false; //marcar la SC como libre
  //si al menos un proceso espera, reactivar al primero
  cola.signal();
end
begin //inicializacion:
  ocupada:=false; //al inicio no hay procesos en SC
end
```



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

El invariante del monitor, es la conjunción de estas dos condiciones:

$$IM :: ocupada == false \Leftrightarrow num_sc == 0 \wedge \\ 0 \leq num_sc \leq 1$$

es decir, no puede ejecutar más de 1 proceso la sección crítica.



Monitores como
mecanismo de alto
nivel

Definición de monitor
Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Demostración del IM:

- Al inicio, IM es cierto (la sección crítica está vacía, luego $num_sc == 0$ y $ocupada == false$).
- Demostración de los procedimientos (`entrar()`, `salir()`) del monitor:
 - Declarar una variable permanente ficticia num_sc (inicialmente $== 0$)
 - Modificar el IM como: $libre + num_sc == 1$ (suponemos la aritmetización de las constantes lógicas)



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

```
procedure entrar();  
{ocupada= false, num_sc= 0 OR ocupada= true, num_sc=  
  1}>{IM}  
if (ocupada) then  
    cola.wait();{Condicion sincronizacion: ocupada  
      == true}  
    num_sc:= num_sc + 1;  
endif;  
{ocupada= true, num_sc= 1}>{IM}  
end;
```

- Demostración del procedimiento *salir()* modificado:

```
procedure salir();  
begin {ocupada== true, num_sc== 1, IM}  
  if cola.queue() then  
    begin {Condicion de sincronizacion}  
      ocupada:= true;  
      cola.signal();  
      {ocupada== true} //condicion de sincronizacion  
    end else begin  
      ocupada:= false;  
      num_sc:= 0;  
      {ocupada== false, num_sc== 0}>{IM}  
    end;  
  {IM}  
end;
```

Sincronización tipo Productor/Consumidor

El problema del Productor-Consumidor con las lecturas y escrituras (E y L) repetidas en un bucle puede solucionarse con el monitor PC, muy sencillo, que encapsula el valor compartido

- El procedimiento `escribir` escribe el parámetro en la variable compartida
- El procedimiento `leer`, lee el valor que hay en la variable

```
process Productor; //calcula x
var a:integer;
begin
  while true do begin
    a:=ProducirValor();
    PC.escribir(a); //copia a en valor
  end
end
process Consumidor //lee x
var b : integer ;
begin
  while true do begin
    PC.leer(b); //copia valor en b
    UsarValor(b);
  end
end
```



Monitores como
mecanismo de alto
nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Monitor Productor/Consumidor

```
Monitor PC ;
var valor_com:integer;//valor compartido
    pendiente:boolean;//true: valor escrito y no leído
    //espera productor hasta que pendiente == false
    cola_prod:condition;
    //espera consumidor hasta que pendiente == true
    cola_cons:condition;
procedure escribir(v:integer);//sentencia E
begin
    if pendiente then
        cola_prod.wait();
        valor_com:=v;
        pendiente:=true;
        cola_cons.signal();
    end;
function leer():integer;//sentencia L
begin
    if (not pendiente) then
        cola_cons.wait();
        result:=valor_com;
        pendiente:=false;
        cola_prod.signal();
    end;
begin // inicializacion }
    pendiente := false ;
end;
```



Monitores como mecanismo de alto nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores



Idea general

Queremos demostrar que el monitor **funciona correctamente** comprobando que cumple un **invariante** en todo momento.

Definiciones

- $\#E$ = número de operaciones de `escribir` completadas.
- $\#L$ = número de operaciones de `leer` completadas.

Invariante del monitor (IM)

$$\#E - \#L = \begin{cases} 0 & \text{si } pendiente = false \quad (\text{no hay dato pendiente}) \\ 1 & \text{si } pendiente = true \quad (\text{hay un dato pendiente}) \end{cases}$$

- Condición del productor: `pendiente == false`
- Condición del consumidor: `pendiente == true`

Conclusión: si este invariante se cumple, el monitor es correcto (porque `pendiente` sólo puede ser `true` o `false`).

Monitores como mecanismo de alto nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores



Demostración del IM

La prueba se hace de manera análoga al caso del **Patrón EM**, sustituyendo:

$$\#E - \#L \equiv num_sec \quad y \quad pendiente \equiv \text{NOT libre.}$$

Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Se cumplen las siguientes condiciones lógicas:

- Al inicio, se cumple el invariante, ya que #E y #L son 0, y pendiente **es** false
- Si se cumple el invariante y se ejecuta escribir:

```
procedure escribir( v:integer );
begin
{pendiente==false,num_escrituras==num_lecturas OR
pendiente==true,num_escrituras==num_lecturas+1 }=> { IM}
  if pendiente then
    cola_prod.wait();
    {Condicion sincronizacion del productor:
      pendiente= false}
    valor_com:=v; //num_escrituras:= num_escrituras+1
    pendiente:=true;
    {Condicion sincronizacion del consumidor:
      pendiente= true}
    if cola_cons.queue() then cola_cons.signal();
    { IM}
  end;
```



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores



- Si se cumple el invariante y se ejecuta leer:

```
function leer():integer;
{pendiente==false, num_escrituras==num_lecturas OR
pendiente==true, num_escrituras==num_lecturas+1 }=> {IM}
begin
  if (not pendiente) then
    cola_cons.wait();
  {Condicion sincronizacion del consumidor:
    pendiente= true}
  result:=valor_com;//num_lecturas:= num_lecturas+1
  pendiente:=false ;
  {Condicion sincronizacion del productor: pendiente
    = false}
  if cola_prod.queue() then cola_prod.signal();
  {IM}
end;
```

Monitores como
mecanismo de alto
nivel

Definición de monitor
Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Idea general

En los monitores, la semántica estándar de las señales **no contempla prioridades**: los procesos bloqueados en una cola se desbloquean siguiendo un orden FIFO (o indefinido).



Monitores como mecanismo de alto nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Extensión con prioridad

```
c.wait(prioridad)
```

- Bloquea el proceso en la cola `c`.
- Los procesos se ordenan según el valor del argumento `prioridad`.

Ejemplo: despertador con prioridades

Varios usuarios indican la hora a la que desean ser despertados. El monitor mantiene la cola ordenada por el valor de la hora (prioridad).



Monitores como mecanismo de alto nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

```
monitor despertador;  
  var  
    ahora: Long_integer;  
    despertar: cond; --prioritaria  
  
  procedure despiertame(n: integer);  
    var alarma: Long_integer;  
  begin  
    alarma:= ahora + n;  
    while ahora< alarma do  
      despertar.wait(n);  
    end do;  
    despertar.signal();  
  end;
```



```
procedure tick(); // cableada a INT CLK
begin
    ahora := ahora + 1;
    despertar.signal();
end;

begin
    ahora := 0;
end;
```

Nota: `ahora` es una variable del sistema que se obtiene procesando la llamada `clock()` del sistema operativo.

Monitores como mecanismo de alto nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores



- Simulación mediante señales FIFO con semántica desplazante

```
Monitor despertador[d:tipo_enumerado]
```

```
var
  ahora:int;
  despertar:cond;
procedure despiertame(n:int)
begin
  alarma:= ahora + n;
  while(ahora< alarma) do
    begin
      despertar.signal();
      despertar.wait();
    end;
  despertar.signal;
end;
procedure tick
begin
  ahora:= ahora + 1;
  despertar.signal();
end;
begin
  ahora:= 0;
end;
```

Escenario de ejecución

id	instruccion	tiempo
P1	despiertame(10)	0
P2	despiertame(2)	1
P3	despiertame(3)	2
P4	despiertame(0)	3

ahora=3

despertar.signal()

inicialmente



Monitores como
mecanismo de alto
nivel

Definición de monitor
Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Todos los mecanismos de señalación alternativos de los monitores



1 SA	señales automáticas	señal implícita
2 SC	señalar y continuar	señal explícita, no desplazante
3 SS	señalar y salir	señal explícita, desplazante, el proceso sale del monitor
4 SE	señalar y esperar	señal explícita, desplazante, el proceso señalador espera en la cola de entrada al monitor
5 SU	señales urgentes	señal explícita, desplazante, el proceso señalador espera en la cola de <i>procesos urgentes</i>

Monitores como mecanismo de alto nivel

Definición de monitor

Funcionamiento de los monitores

Sincronización en monitores

Verificación de monitores

Patrones de solución con monitores

Colas de prioridad

Semántica de las señales de los monitores

Implementación de los monitores

Posibles semánticas de las señales en monitores

Idea clave

Cuando un proceso ejecuta `signal()`, hay varias formas posibles de decidir **qué proceso continúa** y en qué orden.



Semántica [SC] – Señalar y Continuar

- ✓ [SC] El **proceso señalador** continúa su ejecución inmediatamente después de ejecutar `signal`.
- ✓ El **proceso señalado** permanece **bloqueado** hasta poder volver a adquirir la exclusión mutua (E.M.) del monitor.

Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los
monitores
Sincronización en
monitores
Verificación de monitores
Patrones de solución con
monitores
Colas de prioridad
Semántica de las señales
de los monitores
Implementación de los
monitores

Resumen intuitivo

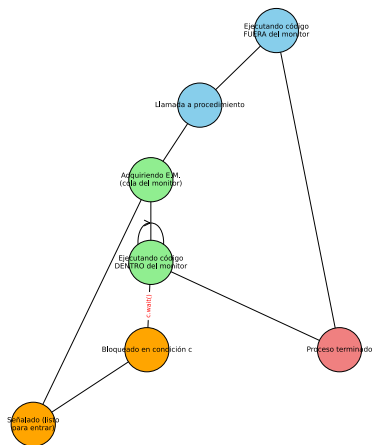
- El señalador no cede la E.M. al proceso señalado.
- El proceso señalado *espera su turno* para entrar en la sección crítica.
- Por eso se denomina: *SC = Señalar y Continuar*.

Señalar y continuar (SC): diagrama de estados del proceso



Monitores como mecanismo de alto nivel

- Definición de monitor
- Funcionamiento de los monitores
- Sincronización en monitores
- Verificación de monitores
- Patrones de solución con monitores
- Colas de prioridad
- Semántica de las señales de los monitores
- Implementación de los monitores



- Señalador: continúa inmediatamente la ejecución de código del procedimiento del monitor tras `signal`
- Señalado: abandona la cola condición y espera en la cola del monitor hasta readquirir la E.M. y ejecutar código tras la operación `wait`

Señalar y continuar (SC): características

El proceso señalador continúa su ejecución dentro del monitor después del signal

- El proceso señalado abandonará después la cola condición y espera en la cola del monitor para readquirir la E.M.
- Tanto el señalador como otros procesos pueden hacer falsa la condición despues de que el señalado abandone la cola condición
- Por tanto, no se puede garantizar que la **condición de sincronización** asociada a **cond** sea cierta al volver **cond.wait()** y, lógicamente, es necesario volver a comprobarla entonces
- Esta semántica obliga a programar la operación wait en un bucle, de la siguiente manera:

```
while not condicion_logica_sincronizacion do  
    cond.wait () ;
```



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores



[SS] El proceso señalado se reactiva inmediatamente

- 2 El proceso señalador abandona el monitor tras hacer `signal`, sin ejecutar el código que haya después de dicho `signal`, la semántica de señal:SS: *señalar y salir* admite varias implementaciones.

Monitores como mecanismo de alto nivel

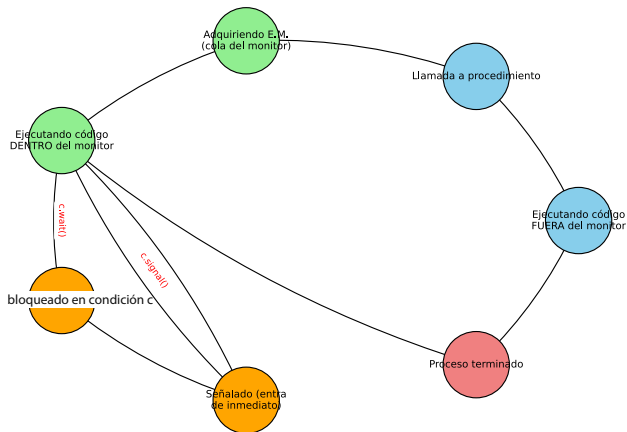
- Definición de monitor
- Funcionamiento de los monitores
- Sincronización en monitores
- Verificación de monitores
- Patrones de solución con monitores
- Colas de prioridad
- Semántica de las señales de los monitores**
- Implementación de los monitores

Señalar y salir (SS): diagrama de estados del proceso



Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los monitores
Sincronización en monitores
Verificación de monitores
Patrones de solución con monitores
Colas de prioridad
Semántica de las señales de los monitores
Implementación de los monitores



- Señalador: abandona el monitor (ejecuta código tras la llamada al procedimiento del monitor). Si hay código en este proceso, tras ejecutar
- Señalado: reanuda inmediatamente la ejecución del código del procedimiento del monitor, programado tras la operación `wait`



El proceso señalador sale del monitor después de ejecutar `cond.signal()`

- Si hay código tras `signal`, no se ejecuta. El proceso señalado reanuda inmediatamente la ejecución de código del monitor.
- En ese caso, la operación `signal` conlleva:
 - Liberar al proceso señalado
 - Terminación del procedimiento del monitor que estaba ejecutando el proceso señalador porque se le obliga a salir del monitor (señales SS)
 - La **condición de sincronización** se mantiene hasta que el señalado continua su ejecución.
 - Esta semántica condiciona el estilo de programación:
 - operación `signal` como última instrucción de los procedimientos
 - o antes de programar una operación `c.wait()`

Monitores como mecanismo de alto nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales de los monitores

Implementación de los
monitores



③ [SE] El proceso señalador se bloquea en la cola del monitor justo después de ejecutar `signal()`

- El proceso señalado entra de forma inmediata en el monitor
- Está asegurada el cumplimiento de la **condición de sincronización**
 - El proceso señalador entra en la cola de procesos del monitor
 - Puede considerarse una semántica injusta respecto del progreso del proceso señalador

Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

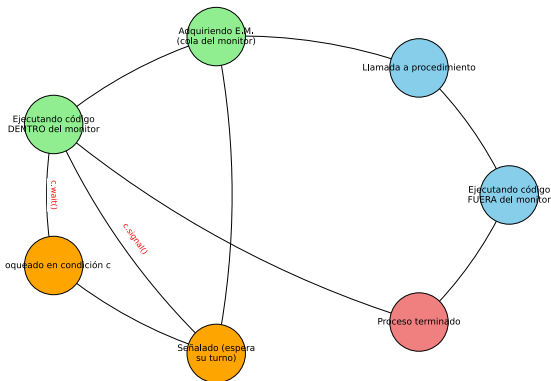
Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales de los monitores

Implementación de los
monitores

Señalar y esperar (SE): diagrama de estados del proceso



- Señalador: se bloquea en la cola del monitor hasta readquirir E.M. y ejecutar el código del monitor tras signal
- Señalado: reanuda inmediatamente la ejecución de código del monitor tras wait



Monitores como mecanismo de alto nivel

- Definición de monitor
- Funcionamiento de los monitores
- Sincronización en monitores
- Verificación de monitores
- Patrones de solución con monitores
- Colas de prioridad
- Semántica de las señales de los monitores
- Implementación de los monitores

Señalar y espera urgente (SU): características



4 [SU] Es similar la semántica SE, pero se intenta corregir el problema de falta de equidad de los señaladores

- El proceso señalador se bloquea justo después de ejecutar la operación `signal()`
 - 1 El proceso señalado entra de forma inmediata en el monitor
 - 2 El proceso señalador entra en una nueva cola de procesos del monitor prioritarios o cola de *procesos urgentes*
 - 3 Los procesos de la cola de procesos urgentes tienen preferencia cuando se queda libre el monitor

Monitores como
mecanismo de alto
nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

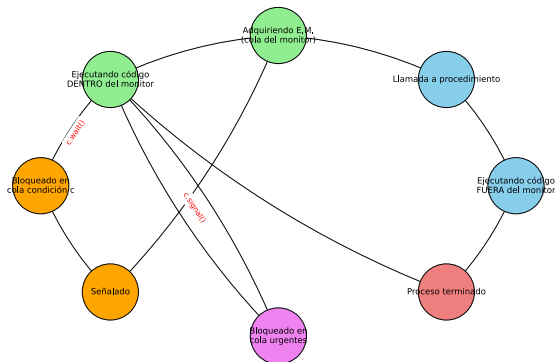
Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Señalar y espera urgente (SU): diagrama de estados del proceso



- Señalador: se bloquea en la cola de urgentes hasta readquirir la E.M. y ejecutar código del monitor tras signal. Para readquirir E.M. tiene más prioridad que los procesos en la cola del monitor
- Señalado: reanuda inmediatamente la ejecución de código del monitor tras la operación `wait`



Monitores como mecanismo de alto nível

Definición de monitor

Funcionamiento de los monitores

Sincronización en monitores

Verificación de monitores

Patrones de solución con monitores

Colas de prioridad

Semántica de las señales de los monitores

Implementación de los monitores

Procesos en la cola de urgentes



Monitores como mecanismo de alto nivel

- Definición de monitor
- Funcionamiento de los monitores
- Sincronización en monitores
- Verificación de monitores
- Patrones de solución con monitores
- Colas de prioridad
- Semántica de las señales de los monitores
- Implementación de los monitores

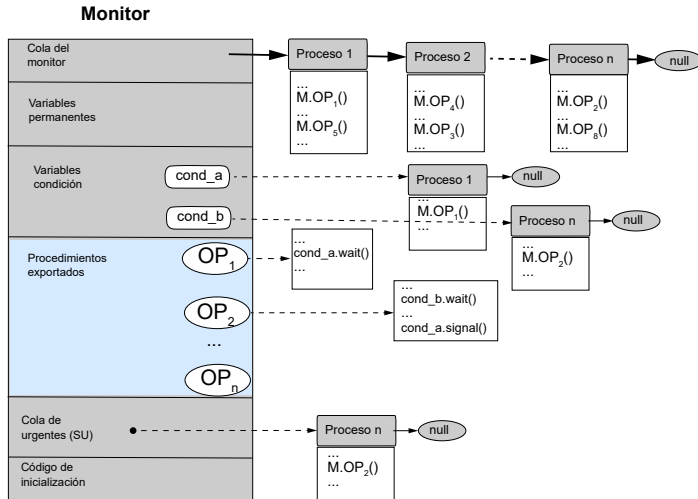


Figura: El proceso n ha ejecutado la op.2, que hace signal en la condición **cond_a**

Análisis comparativo de las diferentes semánticas de señales



Potencia expresiva: todas las semánticas son capaces de resolver los mismos problemas

Facilidad de uso:

La semántica SS condiciona el estilo de programación y puede llevar a aumentar de forma artificial el número de procedimientos

Eficiencia:

- Las semánticas SE y SU resultan ineficientes cuando no hay código tras `signal`
- La semántica SC también es un poco ineficiente al obligar a usar un bucle de comprobación para evitar el *robo de señal*

Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los monitores
Sincronización en monitores
Verificación de monitores
Patrones de solución con monitores
Colas de prioridad
Semántica de las señales de los monitores
Implementación de los monitores

Comparativa entre las distintas semánticas de señales mediante un ejemplo



Barrera parcial

- El monitor tiene un único procedimiento público llamado *cita*
- Hay p procesos ejecutando un bucle indefinido, en cada iteración realizan una actividad de duración arbitraria y después llaman a *cita*
- Ningún proceso termina *cita* antes de que haya al menos n de ellos que la hayan iniciado (donde $1 < n < p$). Después de esperar en *cita*, pero antes de terminarla, el proceso imprime un mensaje
- Cada vez que un grupo de n procesos llegan a la cita, esos n procesos imprimen su mensaje antes de que lo haga ningún otro proceso que haya llegado después de todos ellos a dicha cita (que sea del siguiente grupo de n)

Monitores como mecanismo de alto nivel

Definición de monitor
Funcionamiento de los monitores

Sincronización en monitores

Verificación de monitores

Patrones de solución con monitores

Colas de prioridad

Semántica de las señales de los monitores

Implementación de los monitores

Cuando un proceso comienza a ejecutar la cita, debe esperar hasta que haya otros $n - 1$ que también lo hayan hecho:

- Eso supone usar una variable condición (la llamamos cola)
- Para saber si hay que esperar o no, es necesario saber cuantos procesos han llegado a la cita pero no la han terminado todavía, para ello usamos una variable entera (contador), inicialmente a 0. Al entrar en la cita, debe incrementarse
- Por tanto, la condición lógica asociada a la variable condición cola es `contador==n`
- El proceso que, tras llegar a la cita e incrementar, observa que `contador==n` debe encargarse de que los procesos en espera abandonan dicha espera y terminen cita (puesto que ya se cumple la condición que esperan)



Monitores como
mecanismo de alto
nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Una posible implementación del monitor



Monitores como mecanismo de alto nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

```
Monitor BP //monitor Barrera Parcial }
var cola : condition; //procesos esperando contador==n
    contador : integer; //numero de procesos que han
        llegado a la cita
procedure cita() ;
begin
    contador := contador+1; //registrar un proceso mas que
        ha llegado
    if (contador<n) then
        cola.wait(); //esperar a que haya n procesos
    else begin //si ya hay n procesos ejecutando la cita
        for i := 1 to n-1 do //para cada uno de estos
            cola.signal(); //despertalo
            contador := 0; //volver a poner el contador a 0
        end
        print("salgo_de_cita"); //mensaje de salida
    end
begin //inicializacion del monitor
contador := 0 ; { inicialmente, no hay procesos en cita }
```

Comportamiento del monitor *barrera parcial* para las distintas semánticas de señales



Si llamamos último al último proceso en llegar a la cita de cada grupo de n (el que observa `contador==n`), ocurrirá lo siguiente:

- *Señalar y Continuar*: Los $n - 1$ procesos señalados abandonan el `wait`, pero pasan a la cola del monitor. Los procesos del grupo de la siguiente cita se podrían adelantar.
- *Señalar y Salir*: en este caso, el último proceso abandona el monitor, no pone `contador` a 0.

Monitores como
mecanismo de alto
nivel

Definición de monitor
Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Comportamiento del monitor *barrera parcial* para las distintas semánticas de señales-II



Monitores como mecanismo de alto nivel

Definición de monitor

Funcionamiento de los monitores

Sincronización en monitores

Verificación de monitores

Patrones de solución con monitores

Colas de prioridad

Semántica de las señales de los monitores

Implementación de los monitores

- *Señalar y Esperar*: Similar a la semántica SS, no es correcta tampoco esta solución.
- *Señalar y Espera Urgente*: el último proceso ejecuta los `signals` a los $n-1$ procesos restantes. Entre cada dos de ellos, espera en la *cola de urgentes* a que el proceso recién señalado abandone el monitor.
- Cuando no quedan procesos bloqueados que señalar, el último proceso vuelve a entrar en el monitor y ejecuta la instrucción `contador:=0`, antes que entre otro proceso al monitor.
- La semántica SU de señales hace que esta solución sea correcta

Implementación alternativa del monitor *barrera parcial*



Monitores como mecanismo de alto nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Monitor BP

```
var cola: {\bf cond}ition; //procesos esperando contador==n
    contador: integer; //numero de procesos esperando en la
        cola
procedure cita() ;
begin
    contador:=contador+1; //registrar un proceso mas
        esperando
    if (contador<n) then //todavia no hay n procesos:
        cola.wait(); //esperar a que los haya
    contador:=contador-1; //registrar un proceso menos
        esperando
    print ("salgo_de_la_cita"); //mensaje de salida
    if (contador>0) then //si hay otros procesos en la cola
        cola.signal(); //despertar al siguiente
    end
begin //inicializacin:
    contador:=0; //inicialmente, no hay procesos en la cola
end;
```



Analizamos el comportamiento en todas las semánticas:

- No funciona con la semántica SC.
- Sí funciona con el resto de semánticas (SE,SS,SU).

En general, hay que ser cuidadoso con la semántica en uso, especialmente si el monitor tiene código tras `signal()`.

Generalmente, la semántica SC puede complicar mucho los diseños

Monitores como
mecanismo de alto
nivel

Definición de monitor
Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Axioma de la operación `c.wait()`

$$\{IM \wedge L\} c.wait() \{IM \wedge L\}$$

- La regla permite demostrar la **corrección parcial** de los programas independientemente de la planificación de los procesos de la cola `c`
- No demuestra, por tanto, por tanto ni la propiedad de vivacidad, ni detecta bloqueos

Axioma de las operaciones `c.signal()`, `c.signal_all()`

$$\{P\} c.signal() \{P\}$$



Monitores como
mecanismo de alto
nivel

Definición de monitor
Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

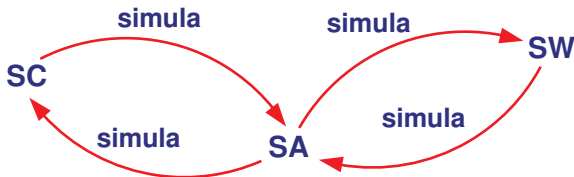
Semántica de las señales
de los monitores

Implementación de los
monitores

Intercambio de señales en programas que usan monitores

condiciones que se han de cumplir para poder intercambiar SW y SC sin modificar adicionalmente el código del monitor:

- 1 Sólo se ha exigir como post**condición** de `c.wait()` el Invariante del Monitor
- 2 Después de una llamada a la operación `c.signal()` se ha de *salir* del monitor
- 3 No se puede utilizar `c.signal_all()`: difusión de una señal a un grupo de procesos



Monitores como
mecanismo de alto
nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Implementaciones alternativas de un semáforo con monitores



Monitores como mecanismo de alto nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

```
Monitor semaforo_FIFO1;
{IM: s>=0}
var c: cond;
    s: int;
procedure P;
begin
    if(s=0) then
        c.wait();
        {s > 0}
        s:=s-1;
    end;
procedure V;
begin
    s:= s+1;
    c.signal;
end;
begin
    s:=0;
end;
```

```
Monitor semaforo_FIFO2;
{IM: s>=0}
var c: cond;
    s: int;
procedure P;
begin
    while(s=0) do
        c.wait();
        {s>= 0}
    end do;
    {s > 0}
    s:=s-1;
end;
procedure V;
begin
    c.signal;
    s:= s+1;
end;
begin
    s:=0;
end;
```

Implementación de los monitores

Esquema de implementación de un monitor con señales SU



Monitores como mecanismo de alto nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

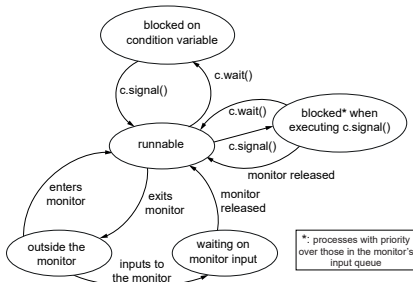
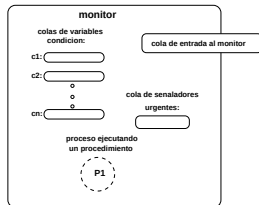
Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores





Concepto fundamental:

- Cola de entrada al monitor: controlada por el semáforo **mutex**
- Cola de procesos urgentes: controlada por el semáforo **next**
- Número de procesos *urgentes* en cola: se contabiliza en la variable **next_count**
- Colas de procesos bloqueados en cada condición: controladas por el semáforo asociado a cada condición **x_sem** y el número de procesos en cada cola se contabiliza en una variable asociada a cada condición (**x_sem_count**)

Monitores como mecanismo de alto nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Implementación de los monitores con semáforos-II



Semáforo **mutex** para implementar la exclusión mutua del monitor

```
procedure P1 (...)  
begin  
  sem_wait(mutex);  
  { cuerpo del procedimiento }  
  sem_signal(mutex);  
end
```

```
//inicializacion  
mutex := 1 ;
```

Monitores como
mecanismo de alto
nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Implementación de los monitores con semáforos-III



Semáforo **next** para implementar la cola de *urgentes* y
next_count para contar los procesos en esa cola

```
procedure P1 (...)
begin
  sem_wait(mutex);
  { cuerpo del procedimiento }
  if (next_count > 0) then
    sem_signal(next);
  else sem_signal(mutex);
end;
```

```
//inicializacion
next := 0 ;
next_count :=0 ;
```

Monitores como
mecanismo de alto
nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Implementación de los monitores con semáforos-IV



Monitores como mecanismo de alto nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores

Para implementar las variables condición: semáforo **x_sem** para cada una y una variable para contar los procesos bloqueados en su cola asociada: **x_sem_count**

```
void entrada() {  
    //implementacion de entrada al monitor  
    sem_wait(mutex); //entre al monitor o se queda bloqueado  
        en la cola de entrada  
}
```

```
void x_wait(Semaphore x_sem, unsigned x_sem_count) {  
    //implementacion de x.wait()  
    x_sem_count := x_sem_count + 1 ;//cuenta 1 proceso mas  
        bloqueado en la cola de condicion "x"  
    if (next_count <> 0) then//hay procesos se aladores  
        esperando  
        sem_signal(next); //desbloquea un proceso se alador  
    else  
        sem_signal(mutex); //deja libre el monitor para que  
            entre otro proceso  
    sem_wait(x_sem); //se bloquea esperando la certeza de  
        condicion "x"  
    x_sem_count := x_sem_count - 1; //cuenta 1 proceso menos  
        bloqueado en la condicion "x"  
}
```

Implementación de los monitores con semáforos-V



Monitores como mecanismo de alto nivel

- Definición de monitor
- Funcionamiento de los monitores
- Sincronización en monitores
- Verificación de monitores
- Patrones de solución con monitores
- Colas de prioridad
- Semántica de las señales de los monitores
- Implementación de los monitores

```
void x_signal(Semaphore x_sem, unsigned x_sem_count){  
    //implementacion de x.signal()  
    if (x_sem_count <> 0) then  
        begin //hay procesos bloqueados esperando la condicion "  
            x"  
            next_count := next_count + 1; //cuenta 1 proceso mas en  
                la cola de se aladores  
            sem_signal(x_sem); //desbloquea 1 proceso esperando:  
                la condicion "x" es cierta ahora  
            sem_wait(next); //entra en la cola de se aladores  
            next_count := next_count - 1; //cuenta 1 proceso  
                se alador menos en cola de se aladores  
        end  
    }  
}
```

```
void salida() {  
    //implementacion de la salida del monitor  
    if (next_count <> 0) then //hay procesos senialadores  
        esperando  
        sem_signal(next); //desbloquea un proceso senialador  
    else  
        sem_signal(mutex); //libera la exclusion mutua del  
            monitor  
    }  
}
```

Ejemplo de uso de la implementación del Monitor

```
Semaphore puede_escribir= 0, puede_leer= 0;
//Monitor PC ;
int valor_com,puede_escribir_count,puede_leer_count;
boolean pendiente;

procedure escribir(puede_escribir,puede_escribir_count);
begin entrada();
  if pendiente then
    x_wait(puede_escribir, puede_escribir_count);
  valor_com:=v; pendiente:=true;
  x_signal(puede_leer, puede_leer_count);
  salida();
end;

function leer(puede_leer,puede_leer_count):integer;
begin entrada();
  if (not pendiente) then
    x_wait(puede_leer, puede_leer_count);
  result:=valor_com; pendiente:=false ;
  x_signal(puede_escribir, puede_escribir_count);
  salida();
end;

begin entrada();
  pendiente := false ;
  valor_com=0;puede_escribir_count=0;puede_leer_count=0;
  salida();
end;
```



Monitores como mecanismo de alto nivel

Definición de monitor

Funcionamiento de los
monitores

Sincronización en
monitores

Verificación de monitores

Patrones de solución con
monitores

Colas de prioridad

Semántica de las señales
de los monitores

Implementación de los
monitores